CSE 221 Project Report

Bran Zhang
UC San Diego

ChaoYuan Lin
UC San Diego

1 Introduction

The goal of this project is to analyze the performance characteristics of the hardware components(CPU, memory, disk, and network) and to understand how these characteristics affect or limit the performance of operating system services on the given machine. Bran is responsible for conducting the CPU, memory and file system performance measurement experiments, while ChaoYuan is responsible for the network performance measurement experiments.

We constructed this project with the following component: a central controller using Microsoft MFC, a number of dynamic libraries (one for each part of the project), and a driver to provide the necessary kernel support. We use Microsoft VS2022 as the IDE for the entire project, the majority of this project is built with the latest VS2022(V143) platform toolset; however, since the visual studio compiler does not provide explicit control over loop unrolling, we switched the dlls to instead use the LLVM (clang-cl) compiler, which is also a builtin option in VS2022. For this project, we use C++17 as the primary programming language to implement our measurement tools. C++17 offers low-level control similar to C while providing modern standard libraries that facilitate development and reduce implementation time. We spent roughly 20 hours implementing and debugging part 1. We expect subsequent parts to take at least as much.

2 Machine Description

In this project, we are running our experiments on one of the most widely used personal computing operating systems in the world – Windows NT. We use a stable version of Windows 10 Pro 21H2(build number 19044.1766) with an Intel 13700KF (Raptor Lake Architecture) processor that supports the x86-64 instruction set. The processor contains in total 16 cores: 8 performance core(P-Core)s and 8 efficient core(E-Core)s with a maximum frequency of 5.4 GHz, 3.4 GHz P-core base frequency and 2.5 GHz E-core base frequency; and 24 hardware threads.

The Processor contains 3 cache levels (L1, L2, and L3) for fast memory access: an 80 KB (32 KB instructions + 48 KB data) L1 P-Core cache and a 96 KB (64 KB instructions + 32 KB data) L1 E-Core; A total of 24MB of L2 cache – the P-Cores have a per-core L2 Cache of 2040KB, while the E-Cores share a L2-cache of (2*4096KB). All cores share a 30 MB L3 cache (Intel Smart Cache). The processor supports a maximum memory bandwidth of 89.6 GB/s.

The Machine is equipped with a dual channel DDR5-4800 DRAM(data rate of 4800 MT/s), which has 2400MHz memory controller frequency, 32768 MB (32 GB) capacity, channel size of 4 * 32bit, and memory bus bandwidth of 4800 * channel size * 1 Byte / 8 bit = 76.8 GB/s.

For the file system, the machine uses two Colorful CN700 pro SSD with 2*2 TB of memory to store all the file data. The disk has a read speed of 7400 MB/s and a write speed of 6600 MB/s for the file system I/O operation [1].

For the I/O bus, the machine uses PCIe 4.0 with a maximum bandwidth of 32 GB/s in a 16-lane slot.

Finally, The machine includes a 2.5 Gb Ethernet network interface for network performance experiments.[2]

Benchmark timing implementation. Since this processor supports invariant tsc at frequency of 3 GHz, all timing we obtained are calculated with the total cycle counts within the measurement duration divided by 3GHz. Note however, since intel does not provide a way to control the CPU frequency, all operations might not run at the theoretical maximum frequency. We exploited Windows' power management control panel to limit the minimal frequency to about 4.7 GHz, however, tasks usually does not run at full 5.4 GHz, which means the timing results we obtained are around 2 4% higher than the theoretical minimum.

[1] We could not find any description on IOPS and latencies for this SSD

[2] The Machine descriptions are obtained from a variety of sources including BIOS, CPU-Z, Intel Website, 3D-Mark

3 CPU, Scheduling, and OS Services

3.1 Timing overhead

Initial Estimate. The read timing overhead occurs mainly because of appropriate memory barriers, or memory fences should be executed in some defined order before and after executing `RDTSC`. This is mainly to avoid out of order execution of the `RDTSC` instruction. Intel processor architectures provide hardware memory fence instructions `MFENCE` and `LFENCE`. The `MFENCE` guarantees that every load and store instruction that precedes the `MFENCE` instruction in program order becomes globally visible before any load or store instruction that follows the `MFENCE` instruction, and the `LFENCE` instruction guarantee an instruction that loads from memory and that precedes an `LFENCE` receives data from memory prior to completion of the `LFENCE` [1]. Since this is only executing 3 instructions, we give an initial estimate and around 1 ns, or only a few cpu cycles.

Method. We construct our timing measurement block by executing the sequence `MFENCE`, `LFENCE` before call to `RDTSC` and `LFENCE` after the return value from `RDTSC` is stored. This sequence of instructions guarantees `RDTSC` to be executed only after all previous instructions have executed and all previous loads and stores are globally visible, and prior to execution of any subsequent instruction (including any memory accesses) as suggested by the `RDTRC` instruction specification in volume 2 of the Intel IA-32 software developer manual [1]. We obtain a single measurement by executing two consecutive blocks to obtain the start *tsc* and end *tsc*, with no instruction in between – effectly measuring the time to executing the instruction sequence `LFENCE`, `MFENCE`, `LFENCE`.

Measurement result. Our results are obtained by recording average and standard deviation over 5,000,000 single measurements, listed in table 1. The overhead is significantly (an order of magnitude of 10) higher than our Initial Estimate. We believe this is because in our initial estimating, we neglected the fact that the memory barriers instructions must block instruction execution until global stores and loads to be visible, which introduces significant overhead since the cpu pipeline essentially stalls when executing the barrier instructions. And global memory loads and stores, even from and to the SRAM takes significantly more time than a few cpu cycles.

	Esti. (ns)	Avg.(ns)	Std (ns)
Timing	1	18.011	0.365

Table 1: Estimate v.s. measured overheads v.s. standard deviation of single read overhead from 0 to 7 arguments over 5000 passes

```
mfence
lfence
call    Asm_RDTSC
mov     rdi, rax
lfence
movzx   eax, byte ptr [rsi+100h]
mov     [rsp+48h+var_19], al
mfence
lfence
call    Asm_RDTSC
```

Figure 1: Disassembly of a read operation from main memory, enclosed by two measurement blocks for obtaining the start and end time stamps., where `Asm_RDTSC` executes the `RDTSC` instruction, reads the results and stores the them in `RAX`

3.2 Loop overhead

Initial Estimate. Loop overhead mainly comes from two sources: (1) a jump instruction that moves the instruction pointer back to the start of the loop, and (2) some arithmetic instructions that evaluates jump conditions and increment the loop counter. Both takes essentially no time, so we give an initial estimate of on the order of 0.01ns, taken prefetching, pipelining into consideration.
Method. A single measurement is obtained by first warming up the branch predictor with an empty loop with only counter increment for 100,000 times. Then the actual measurement is taken again by looping another empty loop 1000 times. We obtain the average and standard deviations over 5000 passes of this measurement.
Measurement result. We obtained an average of 0.255 nanosecond for loop overhead measure by looping over a counter from 0 to 1000 and take the average over 5000 passes. The result is a little less than the time for one CPU cycle, which is very close to our Initial Estimate.

	Esti. (ns)	Avg.(ns)	Std (ns)
Loop	10	0.255	0.256

Table 2: Estimate v.s. measured overheads v.s. standard deviation of loop overhead from 0 to 7 arguments over 5000 passes

3.3 Procedure call overhead

Initial Estimate. Procedural call overhead usually is consisted of a *Call* instruction that redirects the instruction pointer to the function's address, and setting up the stack that stores the parameters, local variables and return values. Relocating the instruction pointer might be the most expensive operation here followed by copying parameter from the caller's stack to the callee's stack. Thus we give the initial estimation of 10ns for procedural call with 0 argument, and an additional 1ns for each additional parameter.
Method. We define 8 procedure functions that take 0 to 7 arguments, respectively. Each function with a positive number of arguments does some stub addition and assignment to the inputs to prevent optimization. The timestamps are recorded

before the procedural call and after the procedural call returns. Before measuring each procedure call, we first call the procedural call 100 times to warm up the cache to obtain consistent results. We obtain the average and standard deviations over 5000 passes of this measurement.

Measurement result. Our measurement from average of 5000 passes gives the following results for 0-7 parameters respectively: 14.494, 14.473, 14.360, 14.327, 14.574, 15.065, 15.211, 15.298. It clearly does not demonstrate the incremental overhead from our initial guess, however, the result is self-explanatory: We overlooked the calling convention when we conducted our Initial Estimate. The *fastcall* convention of X86-84 stores the first 4 arguments in registers *rcx*, *rdx*, *r8*, *r9* respectively, and pushes all subsequent arguments onto the stack. Thus the incremental overhead should only start at procedule call with 5 arguments, which is what the results demonstrate.

	Esti. (ns)	Avg.(ns)	Std (ns)
0 arg	10	14.494	2.985
1 arg	11	14.473	2.884
2 args	12	14.360	2.971
3 args	13	14.327	3.004
4 args	14	14.574	2.968
5 args	15	15.065	2.762
6 args	16	15.211	2.770
7 args	17	15.298	2.891

Table 3: Estimate v.s. measured overheads v.s. standard deviation of procedural call from 0 to 7 arguments over 5000 passes

3.4 System call overhead

Initial Estimate. System calls involves the calling process to trap into the kernel, the kernel performs the actual system call and return the result back to the usermode caller. In Windows, additional overhead is imposed since usually system calls interfaces are exposed to usermode as wrapper functions in a dynamic library(dll). e.g. user calls to NTOpenProcess exported from Ntdll.dll, this function is just a wrapper that validates parameters and make the actual system call by setting up the context and calls the *syscall* instruction. The calling thread then blocks and wait for system call response, Meanwhile, in the kernel system the call handler indexes into the systemcall table and call the actual system call function. This is a lengthy process, even for an ideally minimal system call. We thus estimate that the process takes in the order of microseconds.

Method. For this measurement, we identified a system call *NtGetCurrentProcessorNumber* as illustrated by Figure 2, which only queries some kernel structures for information, and thus only memory reads. We imported the usermode

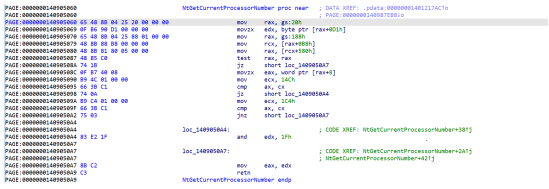

Figure 2: Disassembly of NtGetCurrentProcessorNumber in ntoskrnl.exe

wrapper function from *ntdll.dll*, a dynamic library that provide usermode API of Windows system calls. Similar to procedural call, we first call the system call 100 times to warm up the cache, then we measure the timestamps before and after the system call for a single measure. We obtain the average and standard deviations over 5000 passes of this measurement.

Measurement result. Although the system call we find might not be the minimal, but it is the shortest we could find. Compared to other system call functions that modifies kernel structures and calls other kernel functions, this is as minimal as it gets. Calling to the user mode wrapper also imposes no measurable overhead since an additional call is negligible compared to overheads imposed by memory reads inside the system call. The result is also promising, with an average of 78.034ns over a 5000 pass, way better than our initial estimation, which would more resemble the timing of an average system call.

	Esti. (ns)	Avg.(ns)	Std (ns)
System Call	1000	78.034	14.311

Table 4: Estimate v.s. measured overheads v.s. standard deviation of minimal system call over 5000 passes

3.5 Task creation time

Initial Estimate. Our initial estimation of Kernel thread creation is on the order of micro-seconds. Since Creating the kernel thread must be done via a system call, and this system call likely takes much longer than the minimal system call from previous subsection. Creating a kernel thread at least consists of the following operation: access control, creating thread context, and initializing per-thread data. We can deduce the complexity of the kernel `NtCreateThread` function simply by looking at how many arguments is accepted by the user mode wrapper function.`CreateThread`. Similarly, our initial estimation of process creation is similar, since creating a process is even more complicated than creating a thread: for a start, it requires loading sections of an image file for disk, and it likely requires creation and updates to more kernel structures, thus more other kernel functions to call. We thus reasonably estimate process creation takes on the order of tens or hundreds of micro-seconds.

Method. A single measurement or kernel thread creation is obtained by reading the start timestamp count before a call to `CreateThread` in the parent thread, after CreateThread returns, the parent thread waits on an event from the child thread. The end timestamp is collected as soon as the child thread starts execution, and it is passed back to the parent thread before child thread's exit. We obtain the average and standard deviations over 5000 passes of this measurement. Similarly, A single measurement for process creation is obtained by reading the start timestamp count before a call to `CreateProcess` in the parent process, the parent process passes a pipe handle to the child process as an argument of the call. The parent process then call `ReadFile`, and blocks waiting the child process to write to the pipe. The child process records the end timestamp count as soon as it starts execution, and pass the result back to parent process by a write to the pipe via `WriteFile`, with the handle it received from the parent process. The parent calculates the duration after returning from `ReadFile`.

Measurement result. Our results are shown in table 5. The kernel thread creation time is a magnitude of 10 times higher than our Initial Estimate. This is likely the overhead from OS scheduling, since our initial estimation only accounted for creation time. For process creation time, the average is significantly higher than our initial estimation, we can think of two reasons why this occurs: 1. same as kernel thread, our initial estimation did not take into account the scheduling time, we only estimated the process creation time. 2. We also, significantly underestimated the kernel process creation time, although only a small share of time compared to time spent in the scheduler's wait queue, it likely takes longer to create per process kernel structures.

	Esti. (μs)	**Avg. (μs)**	**Std (μs)**
Process	100	40485.879	1,918.134
KThread	1	23.553	13.512

Table 5: Estimate v.s. measured overheads v.s. standard deviation of process and kernel thread creation overhead over 5000 passes

3.6 Context switch time

Initial Estimate. Context switch between user process typically involves trapping into the kernel for saving the states (i.e. general purpose registers, flags, instruction pointers, etc) of the old process and loading the states of the new process, updating some kernel structures and writing the *cr3* register with the new process's page directory base address, flushing the TLB, and scheduling the main thread of new process to run. Despite the complicated process, With pipelining and multiple logical cores available, it should not take too long – not as long as creating a new process. We thus give an initial estimate of in the order of micro-seconds. In contrast, context switches between kernel threads within the same process should take a lot shorter, since they are share the same process memory space, and there are overall less kernel structures to update, we thus give an initial estimation in the order of 100 nano-seconds – a factor of 10 times less than context switches between processes.

Method. We implemented Context Switch between processes by using the Imbench method [2], which involves passing a token between 20 processes via different pipes. A process blocks until it receives the token from the previous pipe, upon reception, it then writes the token to the next pipe and blocks waiting again. Each internal pass thus consists of 20 context switches. The overhead of loops, reading/writing to pipes or the *nproc* overhead [2], is subtracted from the final result. Note, however, that we did not implement any cache footprint to simulate real-world context switch overhead, but instead measured the theoretical raw context switch overhead. We adapted this methodology to measure kernel threads as well, the only difference in measuring context switch between kernel threads is that instead of passing the tokens around processes, now the tokens are passed between kernel threads. To accommodate for the efficiency of the benchmark, we decided to hard code a single measurement to execute 5 internal passes, or 100 context switches, and a single context switch time the average over 100 context switches, as invoking a single measurement involves all the delays of creating processes/kernel threads and the pipes. The overall average and standard deviation are taken over 50 external passes (50 independent, single measurements), achieving a total of 5,000 context switches.

Measurement result. The average over 5000 passes for process context switch is 3.387 microseconds, close to our Initial Estimate. The average is taken over all 5000 passes, or 100,000 context switches.

	Esti. (μs)	**Avg. (μs)**	**Std (μs)**
Process	1	1032.433	77.669
KThread	0.1	1.828	0.301

Table 6: Estimate v.s. measured overheads v.s. standard deviation of process and kernel thread context switch overhead over 50 external passes, each measures the average of 100 context switches

4 Memory

4.1 RAM access time

Initial Estimate. Since we already know the cache sizes for each level of cache in Section 2's machine descriptions. We expect boundaries roughly at array size of 32KB, 2040KB and 30MB for L1, L2, and L3 cache, respectively, given the process is executed in a P-core. We expect the timing to be around 1 ns for L1 cache (should be similar to the result of a

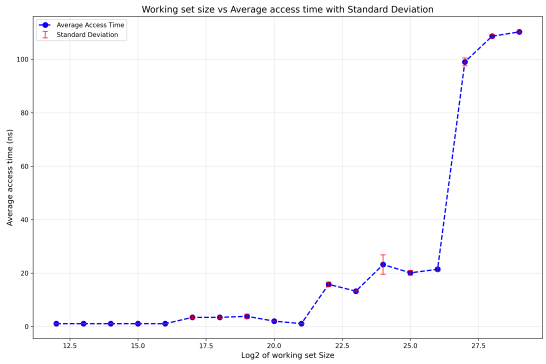

Figure 3: Log 2 of working set from 4 KB to 512 MB v.s. average access time and standard deviation

single read), around 5-10ns for L2 cache access, 10-20ns for L3 cache access, and 50-70ns for main memory access.

Method. We use Imbench's method, with a few modifications for measuring back-to-back load latency [2]:

First, the Imbench paper's benchmark code is compiled in 32-bit mode, our benchmark runs in 64-bit mode. Instead of creating pointer chains consisting of 64-bit pointers, we use an array of 32-bit integers (DWORD), where each element stores the offset to the next array element we want to access. This is effectively the same as dereferencing a pointer chain, since each subsequent memory access depends on the result of the previous memory access, this forces the CPU to serialize memory accesses and defeats out-of-order execution;

Second, to overcome CPU's hardware prefetcher, we have to add some randomness to the pointer chasing, we implement this by pre-loading offsets into the array, with each element storing its own array index, e.g. array[0] stores 0, array[1] stores 1... we then shuffle them in random order. In this way, starting at any index, we guarantee access every element of the array and wraps around–the working set is the entire array.

To further prevent subsequent accesses from reading address in the same cache line of a previous access, the reordering of array index is done at cache line size granularity, since we have a 64 byte cache line, every address we access is divisible by 64. In terms of array elements of 32-bit integers, this means each array index we access is at least 16 apart. For example, we might access the array in the order array[64], array[32], array[16], array[256], all divisible by 16. but we never access elements such as array[1] or array[4].

A single measurement with a given array size involves accessing the array 10 million times (to ensure large enough working sets for larger arrays of more than 30 MB, the L3 cache size). To accommodate for the execution time of this benchmark with large array sizes,we made the decision to take the average and standard deviation of only 10 single measurements for each array size.

Measurement result. The resulting access time from measuring array size of one page (4096 byte, or 1024 integers), all the way to a maximum array size of 512 MB (2^{29} bytes, or 134,217,728 Integers), with each measurement doubling the array size, are shown in figure 3, and the size for each cache level we deduced based our results versus the actual cache sizes is summarized in table 7. Notice L1 and L3 cache size are a magnitude of 2 larger than the size of the actual cache size, we can think of two reasons why this occurred: 1. our working set size are chosen at log 2 resolution, thus no intermediate results were obtained. 2. since we do not flush the cache after preloading the arrays, there might be some initial cache hits that make the resulting timing of a particular working set less than the actual timing. This is a design tradeoff, where if we flush all the array from cache before measurement, we might obtain inaccurate timing, and if we do not flush all caches, the reflected cache size boundary might be inaccurate.

Cache Level	Estimated size	Actual size
L1	64 KB	32 KB
L2	2048 KB	2048 KB
L3	64 MB	30 MB

Table 7: Estimated size of each cache level based on our measurements v.s. the actual cache size specified by Intel

4.2 RAM bandwidth

Initial Estimate. Since we already know that the specific processor we are using supports a maximum memory bandwidth of 89.6GB/s, and the memory bus bandwidth of our DRAM is 76.8GB/s, we given an initial estimate of 70 GB/s for both read and write bandwidth, accounting for the fact that the CPU does not always run at max frequency of 5.4GHz. This maximum should occur when we use 24 threads, since the processor we use supports (16 physical cores + 8 hyper-threads), or a total of 24 logical cores, any additional threads cause 2 threads being scheduled on the same core, and start context switching.

	Esti. (GB/S)	**Avg.** (GB/S)	**Std** (GB/S)
Read	70	44.068	0.764
Write	70	38.399	5.695

Table 8: Estimated v.s. measured average v.s. standard deviation of peak memory read and memory write bandwidth over 50 trials

Method. Memory Bandwidth is measured by creating multiple worker kernel threads. Each kernel thread is scheduled on a distinct logical core if possible, and performs sequential memory access (for read bandwidth) or sequential memory write (for write bandwidth) over a per thread buffer of 256 MB with stride of cache line size of 64 Bytes. Since the total

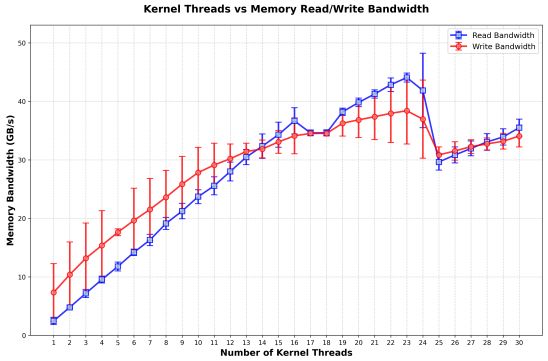

Figure 4: Average Memory Read/Write Bandwidth and Stardard Deviation v.s. number of kernel threads

loop count is determined at compile time, we further optimized the loop by fully unrolling it using compiler specifiers. Sequential memory operations maximize the benefit of cache line prefetching. To prevent threads from context switching, we set each worker thread to the max thread priority. A single Measurement is obtained by measuring the time it takes for all dispatched worker threads to finish their work, the memory bandwidth is the inverse of the time measured, we use this method since we believe this will result in more accurate result than execute all worker threads in a fixed time frame and interrupt them once the time runs out and query the number of bytes they have read/written, since 1. it takes extra instructions and times for the main thread to notify all worker threads to exit, and 2. worker threads must keep track of the number of bytes they read/write. Although there is an indirect method for the main thread to know the number of bytes a thread has written by reading the memory region itself, for read bandwidth, worker threads must keep track of the bytes they read, which would significantly reduce read bandwidth.

Measurement result. The results of average memory read and memory write bandwidths, and corresponding standard deviations over 50 passes versus the number of kernel threads are plotted in Figure 4. The maximum bandwidths, listed in table 8, occurred at actually one less than the number of logical cores the processor has, we suspect this is because the OS reserves most CPU cycles of one of the logical cores for critical background system tasks. Notice that the bandwidth we obtained is nearly half of our initial estimate and the theoretical maximum of the processor, we believe this is as a result of both measurement overhead and DRAM bandwidth limit of our SSD. In our implementation, since we want all threads to start memory operations at the same time, we had to loop through event objects associated with each thread and notify that thread to wake up. Similarly, the timer only stops when all threads notify that they have finished, and those operations add substantial overhead and underestimate the actual memory bandwidth. However, the number of threads to saturate bandwidth should not be influenced by these overheads.

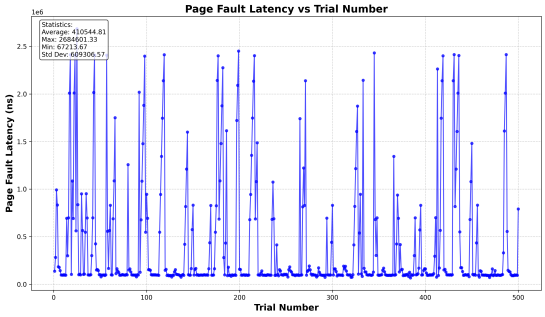

Figure 5: Page fault service latency in nanoseconds for 500 consecutive trials

4.3 Page fault service time

Initial Estimate. Page fault service usually involves triggering an exception, which will be handled by the kernel by fetching the faulting pages from disk memory, after which will be similar to accessing data from main memory. Since reading from SSD disk memory usually takes longer than reading from main memory, we expect it to take around 10 times longer than reading from main memory. We give an initial estimate of 1 μs

Method. To prevent possible caching and aysnchronous cleanup, we first prepare our measurement by creating 500 different 8 KB disk files, each filled with random data, and written to disk. We create our benchmark by calculating average and standard deviation over 500 trials, each accesses a different disk file and creates a different memory mapped file. In each trial, we measure the time it takes to read a single value from the first page of the memory-mapped file, which will trigger a page fault due to demand paging.

Measurement result. We obtained the average and standard deviation over 500 passes, and the results are presented in table 9. The average is about 100 times higher than our initial estimate, with a significant standard deviation. Based on our results, the latency is as low as 70 μs and as high as 2.5 ms, with an average of 500 μs. The number of trials versus latency is plotted in figure 5, despite occasional spikes of 1-2 ms, majority of page fault service latencies are around 50 - 100 μs, only 26% (131/500) of the trials takes longer than average, and only 6.2% (31/500) trials take over 2 ms, contributing to the peaks in figure 6. we believe this is a result of the OS's scheduling decisions prioritizing other background tasks' memory I/O operations, and thus delaying our request to read from disk. We verified that 500 page faults actually occurred during our measurement by using the Windows performance monitor to keep track of the number of page faults, as shown in figure 6. We also underestimated disk read latency in our initial estimate. Based on our results, disk read latency is on the order of 10 μs.

	Esti. (μs)	Avg. (μs)	Std (μs)
Page Fault	1	410.544	609.306

Table 9: Estimated v.s. measured average v.s. standard deviation of page fault service time for 500 passes

5 Network

For the server, we use an AWS EC2 t3.micro instance running Amazon Linux 2023(kernel 6.1, x86-64). The instance provides 2 vCPUs backed by an Intel Xeon Scalable processor on the AWS Nitro virtualization platform, along with 1 GB of DDR4 memory. Storage is provided through an AWS-virtualized NVMe SSD. The instance uses the default EC2 virtual network interface, which offers burstable network bandwidth suitable for lightweight TCP benchmarking.

5.1 Round trip time

Initial Estimate. To estimate the application-level RTT, we exclude the TCP 3-way handshake and focus on the send–echo–receive cycle. Our client machine runs Windows 10 on a modern Intel processor with a fast loopback interface and a 2.5 Gb Ethernet NIC, while the server is an AWS EC2 t3.micro instance located in the us-west-1 region. Because the client is in San Diego and the server is in Northern California, the RTT is dominated by Internet propagation delay rather than CPU or NIC performance. Based on typical latency between these locations, we estimate the remote application-level RTT to be about 25 milliseconds. For localhost on the Windows client, packets stay entirely inside the kernel and avoid the NIC, so we estimate the localhost RTT to be around 50 microseconds. For ICMP ping, packets stay entirely in the kernel network stack. Remote ping RTT reflects only internet propagation delay and network stack processing, so for San Diego to us-west-1 we estimate a ping RTT of about 20 milliseconds. For localhost on Windows, ICMP echo handling is extremely fast and avoids TCP overhead, so we estimate ping RTT on the client machine to be approximately 10 microseconds.

Method. On the server side, we launch an EC2 instance on AWS along with the server code to start the server for our network performance experiments. For the application-level Round Trip Time experiment, we setup our network connection by connecting the client to the server's public IPv4 address, then we send a small packet with 1 byte data over the network to avoid overloading as well as measuring the exact round trip time. We also discard the first sample of our measurement because the first iteration often include some physical setup overhead that have a much higher RRT than any RTT after. For the local Round Trip Time experiment, we have the similar setup as the application-level RTT experiment except we connect the client to the localhost address that is running our local server on the same machine. On the other hand, ping command is already built in the kernel and we can just simply run the ping command on the command prompt/terminal. However, Windows's Ping command doesn't collect the RTT and give us an summary of the RTT's average and standard deviation. Therefore, we use a pipe to run the ping command every time then read all the output into memory and parse the RTT from the output for our application-level ping command experiment. For local ping command experiment, the ping command output $< 1ms$ when the RTT is less than 1 milliseconds which is not sufficient for our measurement. Thus, we implement our own local ping benchmark using ICMP to measure the time from `sendto` to `recvfrom` for the local ping experiment.

Measurement result. We run all the experiments we describe above over 500 passes and the results are present in Table 10 below. For the application-level RTT, our initial estimate for the remote case was around 25 ms, based on the expected wide-area Internet latency between San Diego and the AWS us-west-1 region. The measured average remote RTT is 17.1 milliseconds with a standard deviation of about 1.4 milliseconds. The low standard deviation compare to the average remote RTT indicate that WAN is very stable and the measured average remote RTT is lower than the estimation shows that routers between San Diego and the AWS us-west-1 region are less congested than usual when we run our network experiment. For the localhost RTT, we estimated the RTT on the Windows loopback interface to be on the order of tens of microseconds. The measured average of 29.7 microseconds with a standard deviation of 17.5 microseconds is consistent with this estimate. The larger relative variance on localhost is expected because loopback RTTs are extremely small. For the ping experiment, we estimated that remote ICMP RTT would be around 20 milliseconds. The measured average remote ping RTT was 14.5 milliseconds with a standard deviation of roughly 1.6 milliseconds. This again shows that the actual path between San Diego and us-west-1 is slightly faster than our estimate, while the standard deviation reflects a stable WAN. For localhost ping, we expected an RTT well under 100 microseconds. Our raw socket implementation measured an average of 76.3 microseconds with a standard deviation of 46.2 microseconds. The much higher variability here is partly due to Windows's raw ICMP processing path, which includes additional checks and buffering, as well as sensitivity of short RTT measurements.

	Esti. (μs)	Avg. (μs)	Std (μs)
RTT(Remote)	25000	17124.7	1415.96
RTT(Local)	50	29.681	17.48
Ping(Remote)	20000	14524	1557.38
Ping(Local)	100	76.283	46.165

Table 10: Estimated v.s. measured average v.s. standard deviation of round trip time for 500 passes

5.2 Peak bandwidth

Initial Estimate. To estimate peak network bandwidth, we send a long continuous stream of data over an already-established connection in fixed amount of time. The remote path is limited by the public internet route and the t3.micro's burstable virtual NIC, not by the client's 2.5 GbE interface. Under these constraints, we give an initial guess of about 35 MB/s for remote throughput. For localhost, data never touches the NIC and instead flows through memory copy paths inside the Windows TCP stack. Modern CPUs can sustain tens of gigabits per second internally, so we estimate the localhost network bandwidth to be 5 GB/s.

Method. For the bandwidth experiments, we keep a single TCP connection open and measure throughput in a series of iterations. In each iteration, we start a timer and then repeatedly call `send()` with a large 4 MB buffer, accumulating the total number of bytes successfully written. We run this loop for approximately 5 seconds, then record both the total bytes sent and the actual elapsed time, which is slightly longer than 5 seconds because the loop exits only after the last `send()` call completes. We chose a 5 second measurement window to allows TCP to reach steady state, amortizes the overhead of individual system calls and makes the measurement less sensitive to small timing inaccuracies. We compute the bandwidth for that iteration by dividing the total bits sent by the measured time interval and convert the result to MB/s. We have the same setup for both remote and local bandwidth experiments except the network address is different.

Measurement result. We ran the bandwidth benchmark for both the remote EC2 server and the local loopback interface describe above over 100 passes with 5 seconds for each pass, and the results are summarized in Table 11. The measured remote bandwidth averaged 101 MB/s, significantly higher than our initial estimate of 35 MB/s. This difference arises because our estimate assumed that the t3.micro's burstable virtual NIC and the public Internet path would be the dominant limiting factors. In practice, AWS often allows short periods of substantial bandwidth bursting, and the path between San Diego and us-west-1 might be uncongested, enabling much higher throughput than expected. Furthermore, our client machine has a fast CPU and memory subsystem that can continuously supply data to the TCP stack, allowing the connection to fully utilize the temporary bandwidth credit available on the EC2 instance. For the localhost experiment, the measured throughput averaged 27 GB/s, far exceeding our initial estimate of 5 GB/s. This is expected due to the capabilities of the client hardware. The Intel 13700KF provides exceptionally high memory bandwidth, supported by dual-channel DDR5-4800 DRAM capable of approximately 76.8 GB/s of theoretical throughput. Since localhost traffic never touches the NIC and instead transfers data through kernel memory copy paths, the loopback interface is effectively limited by CPU and DRAM bandwidth rather than physical network hardware.

	Esti. (MB/s)	**Avg.** (MB/s)	**Std** (MB/s)
Remote	35	101.292	13.071
Local	5000	27224.4	434.215

Table 11: Estimated v.s. measured average v.s. standard deviation of peak bandwidth for 100 passes

5.3 Connection overhead

Initial Estimate. To estimate network connection overhead, we measure separately the time required for connect to complete the TCP 3-way handshake and the time required for close to finish the teardown. For the remote case, both phases are dominated by the wide-area RTT between San Diego and AWS us-west-1, since each involves at least one network round trip. Based on our earlier RTT estimate, we guess that the remote connection setup takes about 25 milliseconds and the teardown takes about 10 milliseconds. On localhost, the handshake occurs entirely over the loopback adapter and is limited by CPU and TCP stack processing rather than network delay. We estimate that localhost connection setup takes about 60 microseconds and teardown takes about 40 microseconds.

Method. For the connection setup and teardown experiment, we measure the time required to establish a TCP connection with the server and the time required to close it. On the client side, we repeatedly create a new TCP socket, connect to the server, and then shut the connection down. For the remote case, we connect to the EC2 server's IPv4 address on port 5002. For the local case, we use the loopback address 127.0.0.1 on the same port. In each iteration, we record the connection time as the interval between just before the `connect()` call and immediately after it returns.To measure teardown time, we first call `shutdown(sock, SD_SEND)` to half-close the connection, then call `recv()` to wait for the server to acknowledge the shutdown, and finally call `closesocket()`. The teardown time is measured from immediately before the shutdown sequence to immediately after the socket is closed.

Measurement result We run 500 iterations for both remote and local connection experiments and compute the average and standard deviation after discarding the first sample. The results are shown in Table 12. For the remote connection setup, our initial estimate was 25,000 microseconds, but the measured average was 17,503 microseconds with a standard deviation of 1,416 microseconds. The lower measured time reflects that the actual WAN RTT between San Diego and AWS us-west-1 was on the lower end of our expectation window, and the EC2 network path did not experience heavy congestion during measurement. For teardown, our estimate was 10,000 microseconds, but the measured average was only 53 microseconds. This large difference occurs because TCP teardown does not always require a full WAN round trip, the client-side shutdown often completes immediately. For the localhost experiment, we estimated connection setup to be

about 60 microseconds, while the measured average was 94.5 microseconds with a standard deviation of 44 microseconds. This overhead is higher than the estimate because Windows' TCP stack performs additional socket bookkeeping and loopback routing operations that are not captured in our code. For teardown, we estimated 40 microseconds but measured 18.3 microseconds. Local teardown is faster than expected because once the loopback stack receives the shutdown signal, both FIN exchange and acknowledgment happen entirely within the kernel without any physical transmission delay.

	Esti. (μs)	**Avg. (μs)**	**Std (μs)**
Connection(Remote)	25000	17503.1	1415.96
Teardown(Remote)	10000	53.117	18.483
Connection(Local)	60	94.473	43.915
Teardown(Local)	40	18.273	11.192

Table 12: Estimated v.s. measured average v.s. standard deviation of Connection/Teardown for 500 passes

6 File System

6.1 Size of file cache

Initial Estimate. The file cache size usually varies depending on the size of the system's available DRAM, since our machine has a DRAM size of 32 GB, we give an initial estimation of one half of the DRAM size—-16 GB.

Method. In preparation, we created a large file of 32 GB, each page in this file is filled with random generated bytes, for each file of size `page_cnt` pages, we only access the first `page_cnt` pages of our large 32 GB file. We first pre-generate a random access pattern using the same methodology as the CPU cache benchmark, but at file block (page) size granularity instead of cache line granularity. We then warm up the cache by issuing read request following the random access pattern. Finally, in the main execution phase, we follow the same random access pattern, and record the time stamp before issuing a synchronous read request via ReadFile and after `ReadFile` returns. Each request only reads the first byte of the page. After all read requests are completed, we take the average and standard deviation. [1]

Measurement result. Our result in figure 6 shows the average read time per byte versus the working set file size from 1 GB to 22 GB, with step of 1 GB. The average read time per byte is around 3 μs from 1 GB to 20 GB working set, this is likely a combination of system call overhead and main

[1]Originally, we run the main benchmark 10 passes before taking the average and standard deviation, however, doing so takes too much time (more than 8 hours) for large file sizes over 20 GB. We explored alternative solutions by using multiple threads, but doing so sometimes breaks the random access pattern, results in a portion of file cache hits that pollute our results. Thus we removed the passes from a single measurement. But we did measure multiple times to guarantee reproducibility of our result

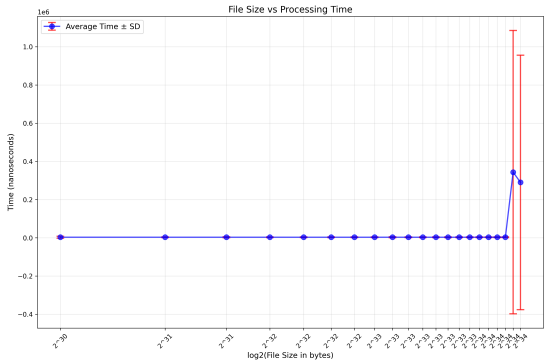

Figure 6: Average per byte read time versus working set disk file size, indicating a file cache size of around 20 GB - 21 GB

memory access. Starting from 21 GB working set, the average read time increased to around 0.3 ms, slightly lower than the average page fault latency of 0.4 ms we obtained in section 4, indicating disk read. The sharp increase in standard deviation is again the result of the OS sometimes queuing up the I/O request. This suggests the file cache size is around larger than 20 GB but less than 21 GB.

6.2 File read time

Initial Estimate. Since we are measuring file read timing without any OS caching, the average read time of random file read should be similar to, or slightly lower than the page fault service time, and the average read time of sequential file read should be lower than random file read.

Method. In preparation, we use the same 32 GB file in our file cache benchmark, with each measurement only accesses the first `page_cnt` pages of the large file. For both sequential and random access, we obtain a file handle by calling `CreateFile` with the `FILE_FLAG_NO_BUFFERING` to prevent the OS from using the file cache. For random access, we first generate a random access pattern of `page_cnt` number of accesses in page size granularity, warm up the instruction cache and TLB by reading the file in this pattern; while for sequential access, we just use a linear access pattern at file block (page) size granularity, Finally, for both operations, we record the start timestamp before issuing a read request by calling `ReadFile`, and collect the end timestamp after `ReadFile` returns and calculate the per page read time. after we collected all timing for reading each page, we take the average and standard deviation.

Measurement result. Figure 7 shows sequential and random average read time and standard deviation from file size of 4 KB to 4 GB. As we can see, the sequential access timing is in general lower than the random access timing. The extremely high standard deviation in some portions of both the sequential and the random plot is again as a result of the OS delaying the synchronous read requests and thus generating

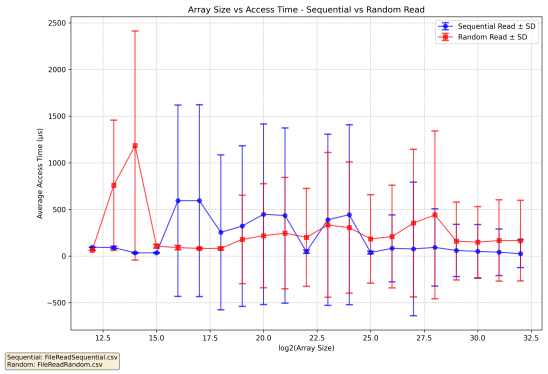

Figure 7: Sequential and Random file read from the local filesystem

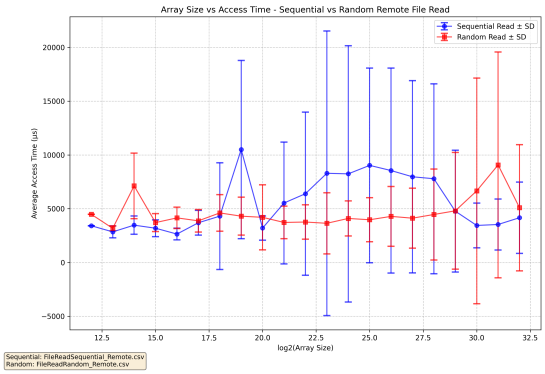

Figure 8: Sequential and random file read from a remote filesystem

exceptionally high read timing. In some parts of the curve, the high standard deviation even causes the sequential read timing to significantly exceed the average random read timing. We performed multiple tries to obtain a better graph, as we cannot control the OS's scheduling decisions, we could not eliminate the high standard deviation. Ideally, without the OS's scheduling delays, the timing should be close to our initial estimate throughout the curves, in the graph shown however, only the portions of the curves with low standard deviation match our initial estimate.

6.3 Remote file read time

Initial Estimate. Reading from a remote file system, even with two machines in the same local area network, must go through the internet protocol layers, and the latency is usually within millisecond range. Other than the additional network latency, the operations happening on the remote machine is exactly the same to that on a local machine. we give an initial estimation of 3-5 milliseconds sdditional network overhead compared to local file reads for both remote sequential file read and remote random file read.

Method. We setup the remote machine (Windows 11 25H2) to use the same WiFi network. The remote machine shares one of its local folder by marking it "sharable", and in our local machine, we map the shared folder as a network drive using the builtin functionality of windows file system. We uses the same large file generated for the file cache benchmark (copied to the remote machine via a USB and into the shared folder) for read measurement. The main benchmark code for sequential and random file read are exactly the same to the benchmark code for local machine, only the path to the file changed. This is because Windows abstracts both remote and local file reading and exposes to the user a single set of API calls: Obtain a file handle via `CreateFile`, and read the file via `ReadFile`, when Windows detects that our path specified is a mapped network driver, it will automatically handle the remote procedure call for us in the background.

Measurement result. Figure 8 shows sequential and random average remote read time and standard deviation from file size of 4 KB to 4 GB. The average remote read timing is around 5-7 ms higher than the local read timing, close to our initial estimation. Although our remote machine uses a newer version of Windows, it appears to share scheduling decisions similar to our local machine. The high standard deviation in some parts of the curves are again the result of OS delaying the synchronous reading request. Ideally without the OS scheduling delays, the remote random read average timing should look only slightly above the remote sequential read average timing, since the 5 ms network delay is significant compared to the 50 - 500 μs average local file read timing.

6.4 Contention

Initial Estimate. Since our goal for this measurement is to show the contention when an increasing number of processes read from the same file, we give an initial estimate of 1 μs per additional process

Method. We reuse the same large 32 GB file as the target disk file, and we define the number of pages each process must read to be `0x1000`. We start by creating k pipes for each of the k child processes. Each child process holds a write handle to a pipe, while the main thread holds all the read handles. The main thread first warms up the cache by creating a random access pattern and read from the target file for three passes, then it creates all child processes, passing the number of pages to read and the write handle to the child process, and blocks waiting to read the results from the child processes. Each child process on execution, opens the target disk file with shared access. set it self to high priority, and read the target file in a randomly generated access pattern, each child process passes the average read time back to the main thread, and finally the main thread calculates the average and standard deviation by collecting timing data from all child processes from the pipes.

Measurement result. We report the main and standard deviation of per-process access time from 1 to 24 process, as

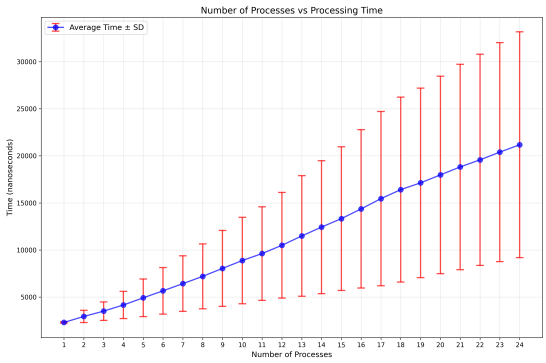

Figure 9: Number of process v.s. the average and standard deviation of per-process access time

my machine supports at most 24 hyper-threads. the result is shown in figure 12. As we expected, the per process access time increases linearly with the number of contenting processes, with the per process average access time over 24 contenting processes process nearly 10 times the average access time of a single process. The increasingly high standard deviation is likely as a result of the race condition between the I/O request from the contenting processes; a process might win some requests but loses the others; the exact order is determined by the OS scheduler.

7 Summary

Operation	Base	Overhead	Pred.	Measured
Timing Overhead	1 ns	0 ns	1 ns	18.011 ns
Loop Overhead	10 ns	0 ns	10 ns	0.255 ns
Proc Call Overhead	10 ns	0 ns	10 ns	15.298 ns
System Call Overhead	1000 ns	0 ns	1000 ns	78.034 ns
Process Creation	100 μs	0 μs	100 μs	40485.879 μs
KThread Creation	1 μs	0 μs	1 μs	23.553 μs
Process Ctx Switch	1 μs	0 μs	1 μs	1032.433 μs
KThread Ctx Switch	0.1 μs	0 μs	0.1 μs	1.828 μs

Table 13: Summary of predicted vs. measured performance across all CPU operations.

Operation	Base	Overhead	Pred.	Measured
L1 Access Time	1 ns	0 ns	1 ns	1 ns
L2 Access Time	7 ns	0 ns	7 ns	7 ns
L3 Access Time	15 ns	0 ns	15 ns	15 ns
DRAM Access Time	60 ns	0 ns	60 ns	60 ns
Read Bandwidth	70 GB/s	0 GB/s	70 GB/s	44.068 GB/s
Write Bandwidth	70 GB/s	0 GB/s	70 GB/s	38.399 GB/s
Page Fault Service	1 μs	0 μs	1 μs	410.544 μs

Table 14: Summary of predicted vs. measured performance across all memory operations.

Finally, make sure you cite all papers that you referenced, for example [2]. Check out this tutorial on how to create a

Operation	Base	Overhead	Pred.	Measured
RTT (Remote)	25000 μs	0 μs	25000 μs	17124.7 μs
RTT (Local)	20 μs	30 μs	50 μs	29.681 μs
Ping (Remote)	20000 μs	0 μs	20000 μs	14524.0 μs
Ping (Local)	20 μs	80 μs	100 μs	76.283 μs
Bandwidth (Remote)	35 MB/s	0 MB/s	35 MB/s	101.292 MB/s
Bandwidth (Local)	5000 MB/s	0 MB/s	5000 MB/s	2722.44 MB/s
Connection (Remote)	25000 μs	0 μs	25000 μs	17503.1 μs
Teardown (Remote)	10000 μs	0 μs	10000 μs	53.117 μs
Connection (Local)	20 μs	40 μs	60 μs	94.473 μs
Teardown (Local)	20 μs	20 μs	40 μs	18.273 μs

Table 15: Summary of predicted vs. measured performance across all network operations.

Operation	Base	Overhead	Pred.	Measured
File Cache Size	16 GB	0 GB	16 GB	20.5 GB
Local File Read (Seq/Rand)	1 μs	0 μs	1 μs	200 μs
Remote File Read	4 ms	0 ms	4 ms	6 ms
Contention (24 Procs)	24 μs	0 μs	24 μs	2000 μs

Table 16: Summary of predicted vs. measured performance across all file system operations.

Biblatex citation.

References

[1] INTEL CORPORATION. *Intel® 64 and IA-32 Architectures Software Developer's Manuals Volume 2*. Intel Corporation.

[2] MCVOY, L., AND STAELIN, C. lmbench: Portable tools for performance analysis. In *USENIX 1996 Annual Technical Conference (USENIX ATC 96)* (San Diego, CA, Jan. 1996), USENIX Association.